第 3 章注意力机制学习教程

基于 ch03/01_main-chapter-code/ch03.ipynb

2026 年 4 月 16 日

目录

1 这份教程解决什么问题 **3**

2 先看全局主线 **3**

3 为什么长序列建模会逼出 attention **4**

3.1 问题的本质 4

3.2 学习时应抓住的转变 4

4 第一步：无可训练参数的 self-attention **4**

4.1 这一步的作用 4

4.2 示例输入 4

4.3 三步计算链 5

4.4 这一步真正要学会的东西 5

4.5 为什么原 notebook 单独讲 naive softmax 6

5 第二步：把 attention 推广到整句 **6**

6 第三步：引入可训练的 Q、K、V 投影 **6**

6.1 为什么前面的版本不够 6

6.2 这三个量各自干什么 6

6.3 为什么要除以根号 d_k 7

6.4 原 notebook 里的两个类 7

7 第四步：为什么语言模型必须使用 causal attention **8**

7.1 问题出在哪里 8

7.2 mask 的几何形状 8

7.3 为什么要在 softmax 前屏蔽 9

7.4 dropout 在这里扮演什么角色 9

7.5 批处理版本的意义 9

8 第五步：从单头到 multi-head attention 9

8.1 为什么不能只用一个头 9

8.2 直观结构图 10

8.3 两种实现思路 10

8.4 这部分真正难的不是公式，而是 shape 10

9 第六步：如何把这一章学成可输出的能力 11

9.1 学完后最应该做的三种输出 11

10 建议的学习节奏 12

10.1 第一遍：只抓主线 12

10.2 第二遍：盯 shape 和 forward 12

10.3 第三遍：做一个小闭环 12

11 本章最短总结 13

1 这份教程解决什么问题

原始 notebook `ch03/01_main-chapter-code/ch03.ipynb` 的目标不是把你变成“会背 attention 公式的人”，而是把你带到一个更实用的位置：

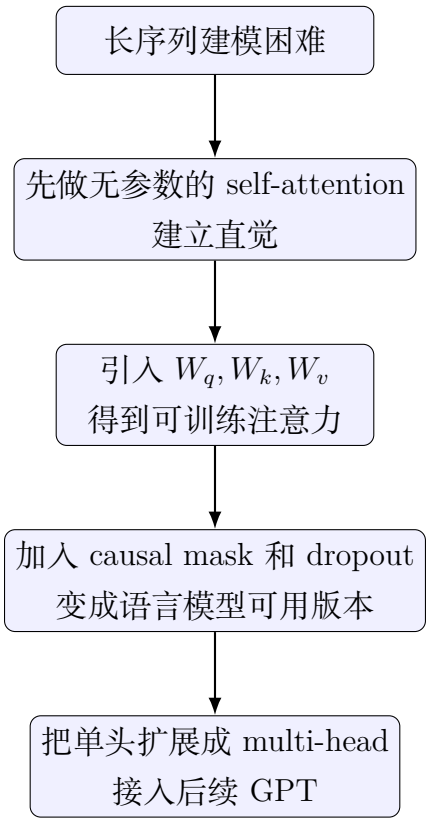
> 你能从零解释 self-attention，能把它写成 PyTorch 模块，能看懂 causal mask，也能理解为什么 GPT 里的注意力最后会变成 multi-head attention。

这份中文教程不逐段翻译原文，而是按学习路径重新组织：

1. 为什么需要 attention；
2. 最小版 self-attention 到底怎么算；
3. 为什么真实模型要引入 W_q、W_k、W_v；
4. 为什么语言模型必须加 causal mask；
5. multi-head attention 究竟比单头多了什么；
6. 学完这一章后，下一步能做什么输出。

2 先看全局主线

这一章的核心不是很多零散知识点，而是一条连续主线：



如果你在阅读时丢了方向，就回到上面这条链：这一章一直在回答“为什么这样设计 attention 模块”，而不只是教你把几行 PyTorch 代码敲出来。

3 为什么长序列建模会逼出 attention

3.1 问题的本质

在早期序列模型里，常见做法是把整句信息压进一个固定大小的隐状态，再靠这个隐状态继续生成输出。这会遇到两个实际问题：

1. 输入一长，前面信息容易被后面过程稀释；
2. 不同输出位置需要依赖输入中的不同部分，但固定向量很难对这种选择性依赖建模。

attention 的想法更直接：

> 当前 token 需要什么信息，就去全序列里按相关性取，而不是指望一个固定摘要把所有信息都无损保存。

3.2 学习时应抓住的转变

从这里开始，你要把自己从“压缩整句”的思维切到“按需检索上下文”的思维。后面的所有公式，本质上都只是在实现下面这句话：

> 给定当前位置，计算它和其他位置的相关性，再用这些相关性对其他位置的信息加权求和。

4 第一步：无可训练参数的 self-attention

4.1 这一步的作用

原 notebook 先故意讲一个**不真实但很适合建立直觉**的版本：直接拿输入向量自己和自己做点积，不引入任何可学习参数。

这样做的好处是，你可以先理解 attention 的计算链，而不被模型参数分散注意力。

4.2 示例输入

notebook 用一句非常短的 toy sentence 做例子：

```
Your journey starts with one step
```

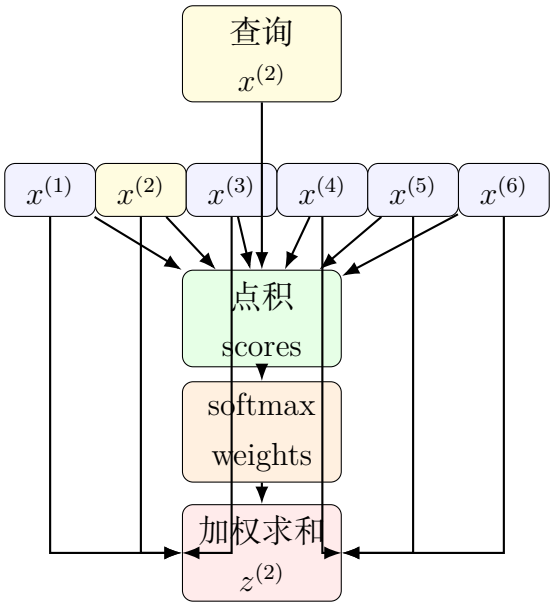
每个 token 先被表示成一个 3 维向量。示例矩阵如下：

token	记号	toy embedding
Your	$x^{(1)}$	$[0.43, 0.15, 0.89]$
journey	$x^{(2)}$	$[0.55, 0.87, 0.66]$
starts	$x^{(3)}$	$[0.57, 0.85, 0.64]$
with	$x^{(4)}$	$[0.22, 0.58, 0.33]$
one	$x^{(5)}$	$[0.77, 0.25, 0.10]$
step	$x^{(6)}$	$[0.05, 0.80, 0.55]$

4.3 三步计算链

假设我们现在只关心第二个 token $x^{(2)}$。这一节的目标是计算它的上下文向量 $z^{(2)}$。

1. 用 $x^{(2)}$ 作为查询，与所有输入向量做点积，得到 attention scores；
2. 对 scores 做 softmax，得到总和为 1 的 attention weights；
3. 用 attention weights 对所有输入向量做加权求和，得到 context vector。



4.4 这一步真正要学会的东西

这一节最重要的不是背住符号，而是把下面三件事说清楚：

1. **attention score** 是相关性的粗分数，还没有归一化；
2. **attention weight** 是 softmax 后的权重，总和为 1；
3. **context vector** 不是复制原 token，而是融合了其他 token 信息的新表示。

也就是说，attention 的输出不是 “把 query 挑出来”，而是 “围绕 query 重新组织一遍整个上下文”。

4.5 为什么原 notebook 单独讲 naive softmax

notebook 先写一个最朴素的 softmax：

```
def softmax_naive(x):
    return torch.exp(x) / torch.exp(x).sum(dim=0)
```

然后再切回 `torch.softmax`。这不是废话，而是在提醒你：

1. softmax 本质上只是把分数转成概率型权重；
2. 真实实现里，数值稳定性和库实现细节非常重要。

5 第二步：把 attention 推广到整句

前一节只算了一个 $z^{(2)}$。但语言模型不会只更新一个位置，它要同时为每个 token 计算上下文向量。

这时就从 “一个 query 对全体 key”，自然推广到 “每个位置都做一次 query”。原 notebook 做了一个很重要的思维跳转：

> 原本看起来像双层循环的计算，其实可以整体改写成矩阵乘法。

于是：

$$\text{attn_scores} = XX^\top, \qquad \text{attn_weights} = \text{softmax}(\text{attn_scores}), \qquad Z = \text{attn_weights}X$$

学习时你要特别注意这一点，因为后面所有高效实现都依赖这个视角：attention 看起来是在两两比较 token，真正落地时则是**张量批量运算**。

6 第三步：引入可训练的 Q、K、V 投影

6.1 为什么前面的版本不够

如果直接用输入向量自己互相点积，模型其实没有学会 “应该用什么标准来判断相关性”。它只是把现成 embedding 空间里的几何关系拿来用。

真实模型需要更强的表达能力，于是引入三组可训练参数：

$$q^{(i)} = x^{(i)}W_q, \qquad k^{(i)} = x^{(i)}W_k, \qquad v^{(i)} = x^{(i)}W_v$$

6.2 这三个量各自干什么

1. Query：当前这个位置正在**找什么**；
2. Key：每个位置手里拿着一张**可被匹配的标签**；

3. Value：真正会被聚合到输出里的**信息载体**。

一个常见误区是把它们当成三份随机复制品。更准确的理解是：

> 同一个输入 x 被投影进了三种不同职责的空间：匹配空间（query/key）和传递信息空间（value）。

6.3 为什么要除以根号 d_k

notebook 在这里引入 scaled dot-product attention：

$$\mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

要点不是记住这个形式，而是理解原因：

1. 维度大时，点积数值会变大；
2. 直接送进 softmax 容易让分布过尖；
3. 过尖会让梯度不稳定，训练效果变差；
4. 所以用 $\sqrt{d_k}$ 做尺度校正。

6.4 原 notebook 里的两个类

`ch03.ipynb` 先写 `SelfAttention_v1`，再写 `SelfAttention_v2`。它们的核心逻辑一致，差别主要在实现风格：

1. `SelfAttention_v1` 直接用 `nn.Parameter` 自己管理权重矩阵；
2. `SelfAttention_v2` 改用 `nn.Linear`，更接近真实工程写法。

其中 `SelfAttention_v2` 的核心非常短：

```
class SelfAttention_v2(nn.Module):
    def __init__(self, d_in, d_out, qkv_bias=False):
        super().__init__()
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key   = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)

    def forward(self, x):
        keys = self.W_key(x)
        queries = self.W_query(x)
        values = self.W_value(x)
```

```
        attn_scores = queries @ keys.T
        attn_weights = torch.softmax(
            attn_scores / keys.shape[-1]**0.5, dim=-1
        )
        context_vec = attn_weights @ values
        return context_vec
```

学习时应该盯住 forward，不是盯住类名。如果你能解释这一段每一行在干什么，这一节就算学到了点子上。

7 第四步：为什么语言模型必须使用 causal attention

7.1 问题出在哪里

前面的 self-attention 默认一个 token 可以看到全序列所有位置。这对编码任务通常没问题，但对自回归语言模型是致命的：

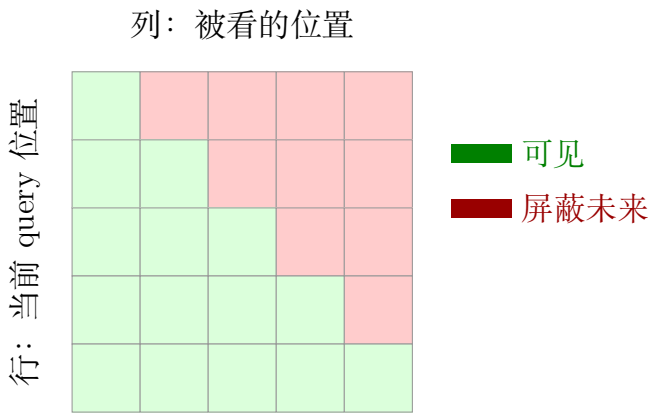
> 如果当前位置在训练时看到了未来 token，模型就等于作弊。

因此，语言模型里的注意力必须满足：

位置 t 只能看见 1 到 t 的 token，不能看见 $t+1$ 之后的 token。

7.2 mask 的几何形状

这就是为什么会出现一个上三角 mask。下面的示意图里，绿色区域表示允许关注，红色区域表示未来位置，必须屏蔽：



7.3 为什么要在 softmax 前屏蔽

原 notebook 先演示了一个直观但不够优雅的方案：先 softmax，再把未来位置乘零，然后重新归一化。

紧接着它说明了更标准的做法：

> 在 softmax 前，把未来位置的 score 直接填成 $-\infty$。

这样 softmax 后这些位置自然变成 0，而且每一行仍然保持合法概率分布。

7.4 dropout 在这里扮演什么角色

dropout 不是 causal mask 的替代品，它解决的是另一个问题：训练时的过拟合。notebook 选择在 attention weights 上施加 dropout，这是常见做法。

你可以把这一步理解成：

> 模型在训练时不要过度依赖某几个固定连接，而要学会更稳健地分配注意力。

7.5 批处理版本的意义

到 `CausalAttention` 这一类时，notebook 不再只处理单句，而是把输入 shape 扩成 $(b, \text{num_tokens}, d)$。

这一点非常关键，因为它标志着代码开始从“教学算例”进入“可接入真实训练循环”的阶段。

8 第五步：从单头到 multi-head attention

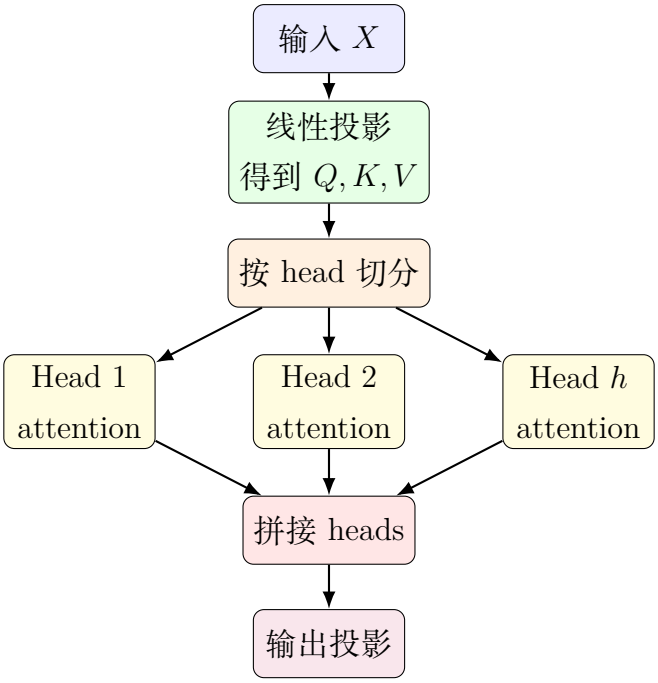
8.1 为什么不能只用一个头

单头 attention 的问题不在于它不能用，而在于它太容易把所有关系都压进一个注意力模式里。multi-head 的核心好处是：

> 让模型在多个子空间里并行地学习不同类型的依赖关系。

比如一个头偏向局部语法，一个头偏向长距离依赖，另一个头偏向指代或结构信息。

8.2 直观结构图



8.3 两种实现思路

原 notebook 先给了 `MultiHeadAttentionWrapper`，它的想法很直观：直接堆多个单头 `CausalAttention`。

然后才给更标准的 `MultiHeadAttention`：

1. 先一次性投影到 d_{out}；
2. 再 reshape 成 $(b, \text{num_tokens}, \text{num_heads}, \text{head_dim})$；
3. 再 transpose 到 $(b, \text{num_heads}, \text{num_tokens}, \text{head_dim})$；
4. 每个 head 独立算 attention；
5. 拼回去，再做输出投影。

8.4 这部分真正难的不是公式，而是 shape

很多人第一次学 multi-head attention 觉得抽象，不是因为公式难，而是因为张量 shape 一直在变。所以学习这一节时，最有价值的训练是把每一步 shape 写出来。

例如 notebook 里的核心流向是：

$$
\begin{aligned}
X &: (b, n, d_{\text{in}}) \\
Q, K, V &: (b, n, d_{\text{out}}) \\
\text{reshape 后} &: (b, n, h, d_h) \\
\text{transpose 后} &: (b, h, n, d_h) \\
\text{scores} &: (b, h, n, n) \\
\text{context} &: (b, h, n, d_h) \\
\text{拼回输出} &: (b, n, d_{\text{out}})
\end{aligned}
$$

只要你把这组 shape 走顺了，multi-head attention 就不会再神秘。

9 第六步：如何把这一章学成可输出的能力

9.1 学完后最应该做的三种输出

输出一：口头解释 你应该能不用看 notebook，独立解释下面这些问题：

1. attention score 和 attention weight 有什么区别；
2. 为什么要有 Q、K、V 三组投影；
3. 为什么要用 $\sqrt{d_k}$ 缩放；
4. causal mask 为什么必须在 softmax 前介入；
5. multi-head 相比单头到底多了什么表达能力。

输出二：最小实现 你应该能在空白文件里手写一个最小版注意力模块：

```
class TinyAttention(nn.Module):
    def __init__(self, d_in, d_out):
        super().__init__()
        self.Wq = nn.Linear(d_in, d_out, bias=False)
        self.Wk = nn.Linear(d_in, d_out, bias=False)
        self.Wv = nn.Linear(d_in, d_out, bias=False)

    def forward(self, x):
        q = self.Wq(x)
        k = self.Wk(x)
        v = self.Wv(x)
        scores = q @ k.transpose(-2, -1)
        weights = torch.softmax(scores / k.shape[-1]**0.5, dim=-1)
```

```
        return weights @ v
```

输出三：接入后续 GPT 这一章的 `MultiHeadAttention` 不是孤立玩具。在仓库后续章节里，它会进入 GPT 模型主体，例如：

1. `ch04/01_main-chapter-code/gpt.py`
2. 多个 `previous_chapters.py` 文件

也就是说，这一章学完以后，你已经具备了理解 GPT block 中注意力子模块的基础。

10 建议的学习节奏

10.1 第一遍：只抓主线

第一遍读 `ch03.ipynb` 时，只做一件事：把整章分成五个阶段记住。

1. 无参数 self-attention；
2. 矩阵化推广；
3. 可训练 $Q/K/V$；
4. causal mask + dropout；
5. multi-head attention。

10.2 第二遍：盯 shape 和 forward

第二遍不要再看大段说明，直接盯几个关键实现：

1. `SelfAttention_v2`
2. `CausalAttention`
3. `MultiHeadAttention`

你的目标不是再看懂一次，而是把每个 `forward` 里的张量流向写在纸上。

10.3 第三遍：做一个小闭环

建议你做这三个动作：

1. 自己手写最小 attention；
2. 打印每一步 shape；
3. 去看第 4 章的 GPT 实现，确认 attention 模块是怎么嵌进去的。

11 本章最短总结

如果把 `ch03.ipynb` 压成几句话，它讲的其实就是：

attention 的本质是：先比较相关性，再按相关性取信息。
self-attention 让每个 token 都能看全局。
引入 $Q/K/V$ 后，这种比较和取信息的方式变成了可学习的。
加入 causal mask 后，attention 才能用于自回归语言模型。
加入 multi-head 后，模型能在多个子空间中并行学习不同关系。
最后，这个模块会直接进入后续 GPT 的实现。

建议的下一步：读完本教程后，回到原 notebook，重点看下面三个实现：

1. `SelfAttention_v2`
2. `CausalAttention`
3. `MultiHeadAttention`

然后自己写一个最小版实现。这样你学到的就不再只是概念，而是可以输出的能力。